MLOps Portfolio: How It Works

Updated 29 October 2026. This guide describes the repaired Python implementation and the execution audit. It replaces the earlier guide's claims about lazy loading, integrated cloud storage and a completed Bedrock demonstration.

1. Explain the project accurately

The project packages a small English spam classifier as a Python HTTP service, sends messages through Kafka for inference, and supplies separate infrastructure exercises. It is an operations demonstration built around a TF-IDF and logistic-regression pipeline. The dataset contains 40 curated examples, not a representative production corpus.

There are five modules. Three reuse the classifier: local serving, Kafka inference and a Kubernetes deployment of the serving image. Terraform separately creates an S3 bucket and a DynamoDB table in LocalStack. These resources are not wired into the application. The optional Bedrock command can ask an LLM to interpret an actual API prediction, but a live AWS invocation is not part of the verified result.

> Analogy: a small practice restaurant has a working kitchen, an order conveyor and a way to open two serving counters. A separately built storeroom exists, but the kitchen does not automatically use it just because it is on the same blueprint.

2. Follow the code and the boundaries

Module	Actual responsibility	Important boundary
`serving/train.py`	Evaluate the fixture, fit the full pipeline, save model and metadata	Evaluation uses held-out folds; the final serving model uses all 40 records
`serving/app.py`	Load the model at startup; expose health, readiness and prediction	No authentication, drift monitor or online retraining
`kafka-streaming/`	Produce messages; call HTTP inference; publish confirmed outputs	At-least-once processing; duplicates remain possible
`kubernetes/`	Run two serving replicas with startup/readiness/liveness probes	Local-cluster demonstration, not a production cloud installation
`terraform-localstack/`	Create an S3 bucket and DynamoDB table in an emulator	No application artifact upload or prediction persistence
`bedrock-explainer/`	Request an LLM interpretation using the API's actual label and score	Requires real AWS access; interpretation is not feature attribution

The root README and `docs/verification.md` record which checks actually ran. The existence of a Dockerfile or YAML manifest alone does not prove a deployment worked.

3. Understand the data and model

The CSV has 40 unique messages: 20 `ham` and 20 `spam`. Its text looks like manually curated demonstration examples; no external benchmark provenance is supplied. The audit does not relabel it as real-world SMS data. There are no exact duplicate texts in this fixture.

`TfidfVectorizer` lowercases text, removes its English stop-word list and learns vocabulary and document-frequency weights. A word that appears everywhere is less informative than one concentrated in a few messages. The resulting sparse matrix feeds logistic regression, which learns weights for a binary decision.

> Analogy: TF-IDF turns each message into a shopping receipt of weighted words. Logistic regression adds up the evidence on that receipt and compares the two possible labels.

A scikit-learn `Pipeline` stores the vectorizer and classifier together. Saving only the classifier would lose the mapping from words to feature positions. `joblib` persists the fitted pipeline; load only an artifact you trust, because pickle-based formats can execute code during deserialization.

4. Separate evaluation from final fitting

The repaired training command runs stratified five-fold evaluation with seed 42. For each fold, the vectorizer and classifier fit only that fold's 32 training messages, then predict the remaining eight. The vocabulary is not fitted once across all 40 before validation.

Fixture evaluation	Measured value
Five fold accuracies	87.5%, 87.5%, 62.5%, 87.5%, 62.5%
Mean accuracy	77.50%
Accuracy standard deviation across folds	12.25 percentage points
Mean spam precision	86.67%
Mean spam recall	65.00%
Mean spam F1	0.7429
Full-fixture training accuracy	100.00%

The standard deviation is fold dispersion, not a confidence interval. These are tiny-fixture diagnostics and do not establish performance on real inboxes. One separate audit-only 32/8 split happened to score 100%; it is not selected as the headline result. That contrast shows how unstable small samples can be.

After evaluation, training fits a fresh pipeline to all 40 records for the serving artifact. `model_metadata.json` records class counts, data/model hashes, seed, scikit-learn version and fold metrics. This report describes the artifact build; it is not a model registry or a deployed monitoring system.

5. Trace an HTTP prediction

The FastAPI lifespan handler loads and checks the trusted model once during process startup. Each replica has its own model instance. The old lazy-load path has been replaced, so the first prediction no longer owns the responsibility for loading the artifact.

`GET /health` reports that the process can answer HTTP. `GET /ready` requires a successfully loaded model. If the artifact is missing or corrupt, the process can remain alive while readiness and prediction return 503. Train the artifact and restart the service to recover; there is no background reload loop.

`POST /predict` accepts exactly one string field, `text`. Blank strings, whitespace-only text, non-string values, extra fields and inputs over 10,000 characters receive 422. The original string is preserved. A text with no known vocabulary tokens still receives the classifier's prior-driven output, with `warning: no_known_tokens` so the limitation is visible.

The response includes text, label, maximum class probability under the legacy field name `confidence`, a model-hash prefix, and an optional warning. That probability is not a calibrated guarantee of correctness. A 0.66 score does not establish that 66 out of 100 similar production messages will be classified correctly.

> Analogy: liveness asks whether the restaurant's phone works. Readiness asks whether the kitchen is equipped to accept an order. A working phone should not make a missing kitchen look ready.

6. Trace one Kafka record

The producer publishes JSON to `messages`. The consumer validates the outer message shape and calls the model API. It retries connection failures, timeouts, 429 and server errors a bounded number of times. A permanent 4xx request failure propagates immediately instead of being retried 40 times.

For input partition 2, offset 17, it constructs `event_id = messages:2:17`. It publishes the prediction with this stable key, waits for the broker acknowledgement through the send future, and only then commits input offset 18 for that specific partition. Kafka committed offsets identify the next record to read.

The old code enabled background offset commits and never inspected the individual publication future. Fault injection showed that an output failure could pass through without being noticed. Regression checks now prove that prediction, send or acknowledgement failure cannot call input commit.

This is at-least-once processing, not exactly-once delivery. If publication succeeds and the process crashes before committing offset 18, restarting can publish the same event again. A downstream deduplicator could use the stable event ID, but no such storage/deduplication implementation exists here. The single-node broker also has replication factor one.

Malformed JSON, invalid input or a persistent API failure stops the consumer without acknowledging the failed record. That preserves retryability, but a poison record can block progress until an operator fixes or handles it. There is no dead-letter queue or automatic recovery supervisor in this demonstration.

7. Containers and orchestration

The serving Dockerfile installs the supported dependencies, copies source and data, trains during the image build and runs Uvicorn on container port 8000. Local examples publish that as host port 8767. `EXPOSE` documents a port; `-p` actually maps it.

Compose starts a real single-node Kafka broker in KRaft mode. Clients inside the network use the advertised address `kafka:9092`; that address is not a ready-made host-client listener. Health-based dependencies keep the inference consumer from starting before the broker and API are ready.

Kubernetes runs two replicas of the same serving image. A startup probe gives model loading time, readiness uses `/ready`, and liveness uses `/health`. The Service selects pods by labels. The acceptance workflow waits for two ready replicas, performs HTTP predictions, deletes a pod and verifies that the Deployment restores capacity.

Containers improve packaging consistency. Floating base-image tags and unresolved transitive dependencies still prevent a promise of bit-for-bit identical rebuilds forever. No throughput, latency SLO, cross-node failover or production capacity claim follows from these functional checks.

8. Terraform and Bedrock

Terraform's locked AWS provider targets LocalStack endpoints with dummy `test` credentials. The lifecycle is init, validate, plan, apply, inspect resources, check a second plan for no changes, then destroy the isolated resources. The S3 bucket is `mlops-model-artifacts`; the on-demand DynamoDB table is `predictions`, keyed by string `id`.

These are provisioned resources, not evidence that the API stores predictions there. LocalStack success is also not an AWS deployment result. The supplied application reads its model from the container filesystem.

The Bedrock CLI now gets the label and score from `/predict`, rather than inventing a 68% example. The request explicitly asks for possible surface clues and identifies its answer as an LLM interpretation. Request-shape tests use a mocked SDK response. No AWS credentials, billable invocation or live Bedrock quality evaluation is included in the passing evidence.

9. Reproduce and explain the result

Start with the root README's Python setup or serving Docker commands. Run `pytest tests -q`, then the integration commands in the module READMEs. CI separates Python regression checks from actual Kafka, Terraform/LocalStack and Kubernetes jobs; a unit-test pass is not substituted for a service integration pass.

For an interview, explain three concrete improvements: model-aware readiness, acknowledge-after-publication streaming, and training-fold-only evaluation. Then state the limits: 40 curated messages, no production accuracy claim, optional unverified Bedrock, independent cloud resources, no monitoring/retraining or exactly-once pipeline.

Sources: Repository and run instructions | Execution verification | FastAPI lifespan | Kafka consumer configuration | scikit-learn common pitfalls